

Forest clustering comparison on Titanic

This notebook compares the original `ForestClusterer`, the three added tree-based estimators, and sklearn baselines on a local Titanic sample.

Algorithms compared:

- `ForestClusterer` - existing random partition embedding implementation.
- `UnsupervisedRandomForestClusterer` - Breiman-style real-vs-synthetic random forest partitioning.
- `ExtraTreesProximityClusterer` - ExtraTrees real-vs-synthetic leaf proximity.
- `UnsupervisedBinaryTreeClusterer` - greedy unsupervised CART-like binary partition tree.
- `KMeans` and `AgglomerativeClustering` - sklearn baselines on the same preprocessed features.

Metrics are smoke/comparison diagnostics, not a claim that Titanic survival is the correct clustering target. `ARI_vs_survived` is included only as an external sanity check.

```
In [1]: from pathlib import Path
import sys

PROJECT_ROOT = Path.cwd().parent if Path.cwd().name == 'notebooks' else Path.cwd().parent
if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.metrics import adjusted_rand_score, normalized_mutual_info_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

from forest_clustering import (
    ForestClusterer,
    UnsupervisedRandomForestClusterer,
    ExtraTreesProximityClusterer,
    UnsupervisedBinaryTreeClusterer,
)

np.set_printoptions(precision=4, suppress=True)
print('Project root:', PROJECT_ROOT)
```

Project root: /mnt/data/work_fc/forest_clustering_patched

```
In [2]: TITANIC_CANDIDATES = [
    Path('/opt/pyenv/lib/python3.13/site-packages/gradio/media_assets/data')
    PROJECT_ROOT / 'data' / 'titanic.csv',
]
```

```

for path in TITANIC_CANDIDATES:
    if path.exists():
        df = pd.read_csv(path)
        print('Loaded Titanic from:', path)
        break
    else:
        raise FileNotFoundError('Local Titanic dataset was not found.')

wanted = ['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare', 'Embarked']
cols = [c for c in wanted if c in df.columns]
X = df[cols].head(300).copy()
y_survived = df['Survived'].head(300).to_numpy() if 'Survived' in df.columns

print('Shape:', X.shape)
display(X.head())
print('Missing values:')
display(X.isna().sum().to_frame('missing'))

```

Loaded Titanic from: /opt/pyvenv/lib/python3.13/site-packages/gradio/media_as
data/titanic.csv

Shape: (300, 7)

	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
0	3	male	22.0	1	0	7.2500	S
1	1	female	38.0	1	0	71.2833	C
2	3	female	26.0	0	0	7.9250	S
3	1	female	35.0	1	0	53.1000	S
4	3	male	35.0	0	0	8.0500	S

Missing values:

	missing
Pclass	0
Sex	0
Age	58
SibSp	0
Parch	0
Fare	0
Embarked	1

```

In [3]: def make_onehot():
        try:
            return OneHotEncoder(handle_unknown='ignore', sparse_output=False)
        except TypeError:
            return OneHotEncoder(handle_unknown='ignore', sparse=False)

numeric_cols = [c for c in X.columns if pd.api.types.is_numeric_dtype(X[c])]
categorical_cols = [c for c in X.columns if c not in numeric_cols]

preprocessor = ColumnTransformer([

```

```

    ('num', Pipeline([('imputer', SimpleImputer(strategy='median')), ('sc
    ('cat', Pipeline([('imputer', SimpleImputer(strategy='most_frequent')
], remainder='drop'))

X_eval = preprocessor.fit_transform(X)
if hasattr(X_eval, 'toarray'):
    X_eval = X_eval.toarray()
print('Evaluation matrix shape:', X_eval.shape)

```

Evaluation matrix shape: (300, 10)

```

In [4]: def safe_silhouette(X_eval, labels):
        unique = np.unique(labels)
        if len(unique) < 2 or len(unique) >= len(labels):
            return np.nan
        return silhouette_score(X_eval, labels)

def model_summary(name, labels, fit_obj=None):
    labels = np.asarray(labels)
    row = {
        'algorithm': name,
        'n_clusters_found': int(len(np.unique(labels))),
        'cluster_sizes': dict(zip(*np.unique(labels, return_counts=True))),
        'silhouette': safe_silhouette(X_eval, labels),
        'calinski_harabasz': calinski_harabasz_score(X_eval, labels) if l
        'davies_bouldin': davies_bouldin_score(X_eval, labels) if len(np.
    }
    if y_survived is not None:
        row['ARI_vs_survived'] = adjusted_rand_score(y_survived, labels)
        row['NMI_vs_survived'] = normalized_mutual_info_score(y_survived,
    if fit_obj is not None and hasattr(fit_obj, 'proximity_matrix'):
        P = fit_obj.proximity_matrix()
        row['proximity_shape'] = str(P.shape)
        row['proximity_diag_mean'] = float(np.diag(P).mean())
    return row

```

```

In [5]: models = []

# Existing library estimator. Use conservative settings so the smoke run
models.append(('ForestClusterer', ForestClusterer(n_iterations=30, n_bins
models.append(('Breiman URF', UnsupervisedRandomForestClusterer(n_estimat
models.append(('ExtraTrees Proximity', ExtraTreesProximityClusterer(n_est
models.append(('Unsupervised Binary Tree', UnsupervisedBinaryTreeClusterer

rows = []
labels_by_name = {}
for name, model in models:
    labels = model.fit_predict(X)
    labels_by_name[name] = labels
    rows.append(model_summary(name, labels, model))
    print(f'{name}: labels={np.unique(labels).tolist()}, sizes={dict(zip(

# sklearn baselines on identical preprocessed features
km = KMeans(n_clusters=3, random_state=123, n_init=10)
labels = km.fit_predict(X_eval)
labels_by_name['sklearn KMeans'] = labels
rows.append(model_summary('sklearn KMeans', labels))

try:
    agg = AgglomerativeClustering(n_clusters=3, metric='euclidean', linka

```

```

except TypeError:
    agg = AgglomerativeClustering(n_clusters=3, affinity='euclidean', linkage='ward')
    labels = agg.fit_predict(X_eval)
    labels_by_name['sklearn Agglomerative Ward'] = labels
    rows.append(model_summary('sklearn Agglomerative Ward', labels))

results = pd.DataFrame(rows)
# Make dict columns readable in both notebook and PDF.
results['cluster_sizes'] = results['cluster_sizes'].map(lambda d: ', '.join(str(v) for v in d.values()))
display(results)

```

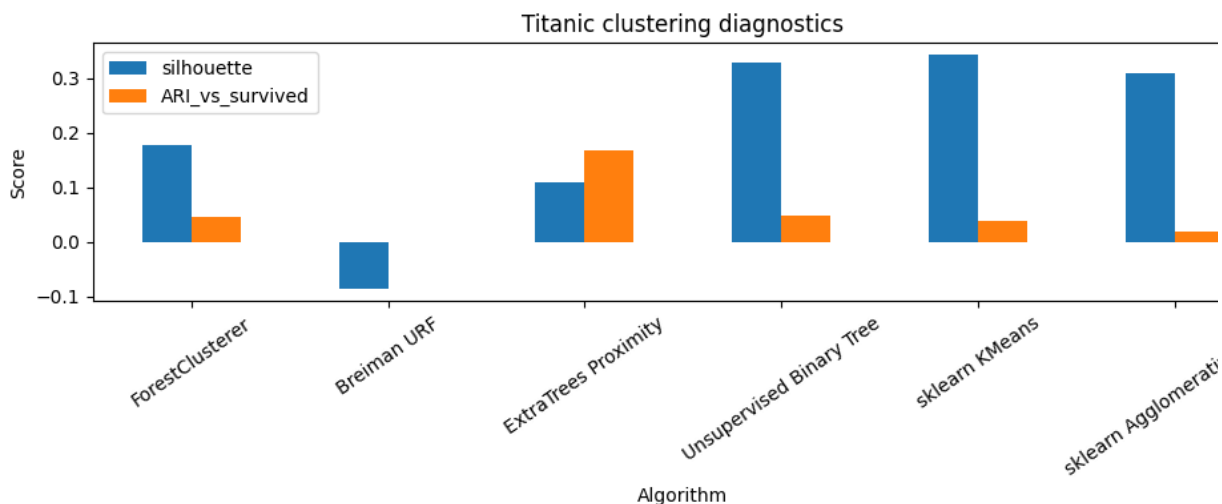
```

ForestClusterer: labels=[0, 1, 2], sizes={np.int32(0): np.int64(114), np.int32(1): np.int64(51), np.int32(2): np.int64(135)}
Breiman URF: labels=[0, 1, 2], sizes={np.int64(0): np.int64(186), np.int64(1): np.int64(2), np.int64(2): np.int64(112)}
ExtraTrees Proximity: labels=[0, 1, 2], sizes={np.int64(0): np.int64(61), np.int64(1): np.int64(147), np.int64(2): np.int64(92)}
Unsupervised Binary Tree: labels=[0, 1, 2], sizes={np.int64(0): np.int64(64), np.int64(1): np.int64(179), np.int64(2): np.int64(57)}

```

	algorithm	n_clusters_found	cluster_sizes	silhouette	calinski_harabasz
0	ForestClusterer	3	0: 1 1 4, 1: 5 1, 2: 1 3 5	0.176647	56.205262
1	Breiman URF	3	0: 1 8 6, 1: 2: 1 1 2	-0.086435	20.528018
2	ExtraTrees Proximity	3	0: 6 1, 1: 1 4 7, 2: 9 2	0.109634	21.215061
3	Unsupervised Binary Tree	3	0: 6 4, 1: 1 7 9, 2: 5 7	0.329706	94.201990
4	sklearn KMeans	3	0: 1 9 2, 1: 6 8, 2: 4 0	0.343431	102.436268
5	sklearn Agglomerative Ward	3	0: 7 8, 1: 5 6, 2: 1 6 6	0.309131	91.042834

```
In [6]: plot_df = results.set_index('algorithm')[['silhouette', 'ARI_vs_survived']]
ax = plot_df.plot(kind='bar', figsize=(10, 4), rot=35)
ax.set_title('Titanic clustering diagnostics')
ax.set_xlabel('Algorithm')
ax.set_ylabel('Score')
plt.tight_layout()
plt.show()
```



```
In [7]: # Pairwise ARI between algorithm outputs: useful to see whether methods p
names = list(labels_by_name)
ari_matrix = pd.DataFrame(index=names, columns=names, dtype=float)
for a in names:
    for b in names:
        ari_matrix.loc[a, b] = adjusted_rand_score(labels_by_name[a], labels_by_name[b])
display(ari_matrix.round(3))
```

	ForestClusterer	Breiman URF	ExtraTrees Proximity	Unsupervised Binary Tree	sklearn KMeans	sklearn Agglomerative Ward
ForestClusterer	1.000	0.071	0.261	0.465	0.352	0.374
Breiman URF	0.071	1.000	0.035	0.124	0.083	0.132
ExtraTrees Proximity	0.261	0.035	1.000	0.044	0.014	0.015
Unsupervised Binary Tree	0.465	0.124	0.044	1.000	0.739	0.799
sklearn KMeans	0.352	0.083	0.014	0.739	1.000	0.705
sklearn Agglomerative Ward	0.374	0.132	0.015	0.799	0.705	1.000

```
In [8]: bt = [m for name, m in models if name == 'Unsupervised Binary Tree'][0]
rules_df = pd.DataFrame(bt.rules())
display(rules_df)
```

leaf	n_samples	rule
0	64	num__Pclass <= - 1.07449
1	179	num__Pclass > - 1.07449 and num__Parch <= 0.142731
2	57	num__Pclass > - 1.07449 and num__Parch > 0.142731

Interpretation

The tree-proximity estimators expose a proximity/distance API, while sklearn baselines cluster directly in the preprocessed feature space. The binary tree is the most interpretable of the new methods because each cluster is defined by an explicit root-to-leaf rule. The RF/ExtraTrees methods usually provide richer proximities because each sample pair is compared across many leaf partitions rather than a single terminal leaf.